

Lab1 report 112550018 吳欣澤 BigPatric

#Describe

Step1 Environment:

Setting up the environment , I use vscode(local) to complete coding and use virtual box to execute files, using git method (e.g pull, add, push).

Install wireshark application on local.

Step2 Topo:

By observing the original code(topo.py), I find the use of function to create topo

Creating Hosts	<code>self.addhost("host name")</code>
Creating Switches	<code>self.addswitch("switch name")</code>
Creating Links	<code>self.addLink("src", "dst", bandwidth)</code>
Record Packets	<code>h2.cmd("tcpdump -w ../out/h2_output.pcap &")</code>
Create flow (h1->h2)	<code>h2.cmd("iperf -s -i 1 -t 5 -p 7777 > ../out/result_s.txt &")</code> <code>h1.cmd("iperf -c " + str(h2.IP()) + " -i 1 -t 5 -p 7777 > ../out/result_c.txt &")</code>

Creating new topo:

Add host : h3,h4

Add switch : s3

Adjust links : h2-s1, s1-s2, s1-s3, s2-h3, s3-h4

(1) topo_TCP.py

Goal : Two TCP from h1->h3,One TCP from h2->h4

```
# TCP flow
h3.cmd("iperf -s -i 1 -t 5 -p 8889 > ../out/TCP_s_h3_1.txt &")
h1.cmd("iperf -c " + str(h3.IP()) + " -i 1 -t 5 -p 8889 > ../out/TCP_c_h1_1.txt &")
h3.cmd("iperf -s -i 1 -t 5 -p 7777 > ../out/TCP_s_h3_2.txt &")
h1.cmd("iperf -c " + str(h3.IP()) + " -i 1 -t 5 -p 7777 > ../out/TCP_c_h1_2.txt &")
h4.cmd("iperf -s -i 1 -t 5 -p 7777 > ../out/TCP_s_h4.txt &")
h2.cmd("iperf -c " + str(h4.IP()) + " -i 1 -t 5 -p 7777 > ../out/TCP_c_h2.txt &")
```

h1->h3 twice and h2->h4 once

In h1->h3 , use different port to create two different flow(8889&7777)

(2) topo_UDP.py

Goal : same like TCP but use UDP flow

=>while creating the flow(iperf), plus "-u" to send UDP

iperf commands

In a iperf commands , there are different commands parameters, which stands for different commands. I will use the iperf commands from UDP as example.

```
# UDP flow
h3.cmd("iperf -s -u -i 1 -t 5 -p 8889 > ../out/UDP_s_h3_1.txt &")
h1.cmd("iperf -c " + str(h3.IP()) + " -u -i 1 -t 5 -p 8889 > ../out/UDP_c_h1_1.txt &")
h3.cmd("iperf -s -u -i 1 -t 5 -p 7777 > ../out/UDP_s_h3_2.txt &")
h1.cmd("iperf -c " + str(h3.IP()) + " -u -i 1 -t 5 -p 7777 > ../out/UDP_c_h1_2.txt &")
h4.cmd("iperf -s -u -i 1 -t 5 -p 7777 > ../out/UDP_s_h4.txt &")
h2.cmd("iperf -c " + str(h4.IP()) + " -u -i 1 -t 5 -p 7777 > ../out/UDP_c_h2.txt &")
```

each flow contain two lines

(1)h3.cmd()

-s	Server mode
-u	Using UDP(no need If TCP)
-l 1	the interval between periodic bandwidth reports to 1 second
-t 5	Duration of test 5 seconds
-p 8889	Port number to listen(8889)

(2)h1.cmd()

Have 2 more commands comparing to h3.cmd

-c	Client mode
Str(h3.IP())	retrieves the IP address of host h3 and converts it to a string.

Step3 computeRate.py

To compute Rate, we need **sending time** and **total size of packets**

From parser.py , we can get path of pcap and know how to read cap and the packetsize using len(). To read 4 pcaps at same time , I use sequence of argv [] to read pcap files,
one by one, like this:

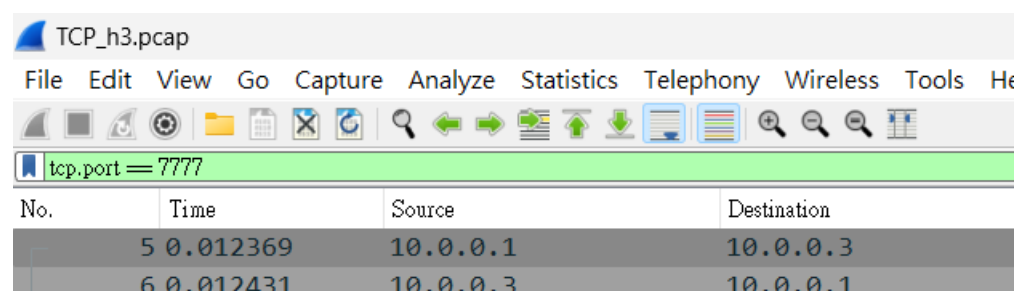
```
cn2023-lab1@cn2023lab1-VirtualBox:~/lab1-BigPatric/src$ sudo python3 computeRate.py  
../out/TCP_h3.pcap ../out/TCP_h4.pcap ../out/UDP_h3.pcap ../out/UDP_h4.pcap
```

For time, use (the time of last sent packet) – (the time of first sent packet).

For packet size, since the results of len() is counted in Bytes, but the answer we want is Mbps(Mega bits per sec), so I times the accumulated len with 8(1 byte = 8 bits),and divide by 10^6(1 mega = 10^6) to get the final answer.

Step4 Wireshark

To verify the answer we can use Wireshark to open pcap files, and filter packets to check whether the answer is right or not. After Step 2, we can get pcap files for each different flow , now I will take TCP_h3.pcap as example to show how to look up the throughput in Wireshark.



No.	Time	Source	Destination
5	0.012369	10.0.0.1	10.0.0.3
6	0.012431	10.0.0.3	10.0.0.1

In tcp_h3, we have two TCP flow with different port, one of the port is 7777, and the other is 8889, so I use the filter command (tcp.port == 7777) or (tcp.port == 8889)

The tcp is filtering the protocol, and the port is its port number, and use == num to filter packets flow from different flow.

Then check out “Statistics” and find the display part.

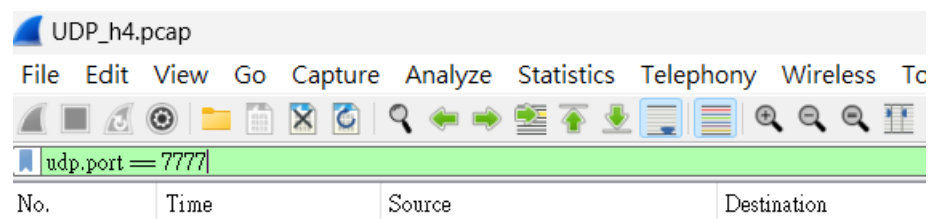
Statistics

Measurement	Captured	Displayed
Packets	918	425 (46.3%)
Time span, s	8.129	6.168
Average pps	112.9	68.9
Average packet size, B	1389	1487
Bytes	1275117	631858 (49.6%)
Average bytes/s	156 k	102 k
Average bits/s	1254 k	819 k

In the Displayed part , the Average bits/s is the throughput we want.(flow 2),the answer we got is pretty close to the one from Wireshark

```
--- TCP ---
Flow1(h1->h3):0.8212684230708866535176355043      Mbps
Flow2(h1->h3):0.8178822509113443562988335763      Mbps
Flow3(h2->h4):1.632828278283887127026675512      Mbps
--- UDP ---
Flow1(h1->h3):0.6074906606213480634758453335      Mbps
Flow2(h1->h3):0.6101727385931862447715444299      Mbps
Flow3(h2->h4):1.088790263952747123808970837      Mbps
```

Then trying to check the UDP part, we have only one flow, so only have to filter one time. Take UDP_h4 as example.



The only difference from TCP is we use “udp” to filter, and the rest steps are usually same.

Statistics

Measurement	Captured	Displayed
Packets	518	443 (85.5%)
Time span, s	8.976	4.960
Average pps	57.7	89.3
Average packet size, B	1311	1512
Bytes	678888	669816 (98.7%)
Average bytes/s	75 k	135 k
Average bits/s	605 k	1080 k

###The UDP problem

At first, while I’m checking the results from my computeRate.py and WireShark, except UDP_h4, the rest of them are quite close to the number from wireshark.

The original answers:

```

--- TCP ---
Flow1(h1->h3):0.8212684230708866535176355043      Mbps
Flow2(h1->h3):0.8178822509113443562988335763      Mbps
Flow3(h2->h4):1.632828278283887127026675512      Mbps
--- UDP ---
Flow1(h1->h3):0.6074906606213480634758453335      Mbps
Flow2(h1->h3):0.6101727385931862447715444299      Mbps
Flow3(h2->h4):0.6170956185445973179324063571      Mbps
cn2023-lab1@cn2023lab1-VirtualBox:~/lab1-BigPatric/src$

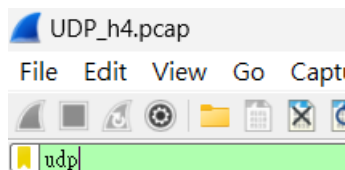
```

And the results from Wireshark:

//TCP_h3_1 #port==8889	<u>Displayed</u> 421 (45.9%) 6.137 68.6 1507 634474 (49.8%) 103 k 827 k
//TCP_h3_2#port==7777	<u>Displayed</u> 425 (46.3%) 6.168 68.9 1487 631858 (49.6%) 102 k 819 k
//TCP_h4#port==7777	<u>Displayed</u> 825 (91.9%) 6.113 135.0 1512 1247590 (99.3%) 204 k 1632 k
//UDP_h3_1#port==8889	<u>Displayed</u> 460 (46.2%) 9.031 50.9 1498 689066 (49.6%) 76 k 610 k
//UDP_h3_2#port==7777	<u>Displayed</u> 460 (46.2%) 9.035 50.9 1502 690910 (49.7%) 76 k 611 k

//UDP_h4#port==7777	<u>Displayed</u> 443 (85.5%) 4.960 89.3 1512 669816 (98.7%) 135 k 1080 k
---------------------	---

In TCP , the difference between mine and Wireshark is quite close, but the at UDP_h4, the difference are quite big, so I tried to find the problem. When I look at the UDP_h4.pcap in Wireshark, I found out that there are something while filtering.

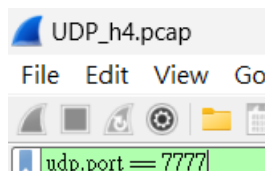


when I only use udp as the filter command , the result I get is

Displayed
473 (91.3%)
8.752
54.0
1427
675078 (99.4%)
77 k
617 k

which is actually same as the original answer. However,

when I add port to the command,



the result I get, which supposed to be the correct answer is:

Displayed
443 (85.5%)
4.960
89.3
1512
669816 (98.7%)
135 k
1080 k

The difference between these two conditions, is that at the former one, there are something at the end:

508 8.605865	fe80::a4ab:32ff:fed... ff02::fb	MDNS	215 Standard query response 0x0000 PTR, cache flush cn2023
511 8.648583	fe80::4cfc:91ff:fe8... ff02::fb	MDNS	215 Standard query response 0x0000 PTR, cache flush cn2023
515 8.708937	fe80::a4ab:32ff:fed... ff02::fb	MDNS	107 Standard query 0x0000 PTR _ipps_tcp.local, "QM" quest
516 8.715940	fe80::4cfc:91ff:fe8... ff02::fb	MDNS	107 Standard query 0x0000 PTR _ipps_tcp.local, "QM" quest
517 8.751681	fe80::64b3:11ff:fe6... ff02::fb	MDNS	107 Standard query 0x0000 PTR _ipps_tcp.local, "QM" quest

The end part of the packets is something using MDNS(Multicast DNS),which indeed affect the throughput, since we should not count those while doing calculation. And so, I changed the way counting the time for UDP_h4,

```
allP = 0
for packet in packets_UDP_h4[UDP]:
    allP+=1
## MDNS is form back
## counting from back, when we find first UDP(the last time spot)
end = 0
for i in range(allP-1,0,-1):
    if packets_UDP_h4[UDP][i].dport == 7777 or packets_UDP_h4[UDP][i].sport == 7777:
        end = i
        break
```

I use allP to count the total amount of packets, and then counting from the back ,the first time I met UDP is the last of that flow , and so I can have the correct time without the ones which are MDNS. In the end , I can get the closer answer to Wireshark.

```
--- TCP ---
Flow1(h1->h3):0.8212684230708866535176355043      Mbps
Flow2(h1->h3):0.8178822509113443562988335763      Mbps
Flow3(h2->h4):1.632828278283887127026675512      Mbps
--- UDP ---
Flow1(h1->h3):0.6074906606213480634758453335      Mbps
Flow2(h1->h3):0.6101727385931862447715444299      Mbps
Flow3(h2->h4):1.088790263952747123808970837      Mbps
```

#Observations from results

At the topo part, while we are creating the links, we send the bandwidth equals to 2.

Which means

(1) The TCP_h3 flows should share the path and so their throughput should be

$bw/2(1)$, but the results I have is 0.821 & 0.817

(2) The rate of TCP_h4 should be equal to bw, but I only get 1.6

(3) The results from UDP are also quite far from the bottleneck throughput.

There is another interesting thing is that all of those UDP flows are smaller than the mapped in TCP, I would try to guess that the reason is because the possibility of packet loss is higher in UDP than TCP, which really affect the throughput.

#What I learned and difficulties

In this homework , I have learned more about the transport layer and how it works.

Like mininet, making us know more about transporting packets, like ports. I also learn about how to use Wireshark and know much more things about what does it do and the filter command. In addition , the ability of reading python code and trying to recreate in my own style. And there is one more thing which is quite important is the Github token. While I am trying to git pull or push in virtual box, it request the username and password, after trying again and again , I finally found out that Github had replace the request of password with personal token.